1 Final Project

Choose one of the following two problems.

1.1 Project A: Architectural Analysis of DeepSeek-V3.2

This project involves a comprehensive explanation of the architecture of DeepSeek-V3.2. You must explain the provided system in full technical detail, highlighting its key innovations. see `https://arxiv.org/pdf/2512.02556`, and `https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp/tree/main/inference`.

Required Components:

(1) DeepSeek Sparse Attention (DSA) and the Lightning Indexer:

- Explain the prototype of DSA, which consists of two main components: a *lightning indexer* and a *fine-grained token selection mechanism.*
- Detail the function of the **lightning indexer**. Its purpose is to compute an index score $I_{t,s}$ between a query token $\mathbf{h}_t$ and preceding tokens $\mathbf{h}_s$ to determine which tokens should be selected for attention. The score is calculated as:

$$I_{t,s} = \sum_{j=1}^{H^l} w_{t,j}^I \cdot \mathrm{ReLU}\left(\mathbf{q}_{t,j}^I \cdot \mathbf{k}_s^I\right),$$

  where H^l is the number of indexer heads, and $\mathbf{q}_{t,j}^I$, $w_{t,j}^I$, and $\mathbf{k}_s^I$ are derived from the query and key tokens. ReLU is used for activation to improve throughput.
- Describe the **fine-grained token selection mechanism**. This component retrieves only the key-value entries $\{\mathbf{c}_s\}$ corresponding to the top-k index scores. The final attention output $\mathbf{u}_t$ for token $\mathbf{h}_t$ is then computed using standard attention on this sparse set:

$$\mathbf{u}_t = \mathrm{Attn}(\mathbf{h}_t, \{\mathbf{c}_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}).$$

- Clarify how DSA is instantiated within the framework of **Multi-Head Latent Attention (MLA)** (from DeepSeek-V2). For computational efficiency, DSA is implemented based on the Multi-Query Attention (MQA) mode of MLA, where a single latent key-value vector is shared across all query heads.
- Explain the two-stage continued pre-training process used to integrate DSA into the base model (DeepSeek-V3.1-Terminus):
  (a) **Dense Warm-up Stage**: All model parameters are frozen except for the lightning indexer, which is trained for 1000 steps using a KL-divergence loss to align its output distribution with that of the main model's dense attention.
  (b) **Sparse Training Stage**: The fine-grained token selection is activated, and all model parameters are optimized to adapt to the sparse attention pattern. The indexer is trained with a loss calculated only over the selected top-k tokens.
- Discuss the efficiency gains: DSA reduces the core attention complexity from $O(L^2)$ to $O(Lk)$ (where $k \ll L$), leading to significant inference speedups for long-context sequences.

(2) The Mixture-of-Experts (MoE) Architecture:

- Describe the core principle of the **DeepSeekMoE** architecture used in DeepSeek-V2, which is a foundational component of the V3 series.
- Highlight its key features designed for economical training and efficient inference:
  - **Fine-grained Expert Segmentation**: Experts are divided into smaller, more numerous units to allow for more specialized and flexible routing.
  - **Shared Expert Isolation**: A subset of experts is designated as "shared" and is always activated, ensuring stability and retaining common knowledge.
  - **Load Balancing Mechanisms**: Strategies like auxiliary loss and token-dropping are employed to ensure experts are utilized evenly during training.

- Specify the scale: DeepSeek-V2 has 236B total parameters but activates only 21B per token, dramatically reducing computational costs during inference compared to dense models of similar total size.
- Explain how this MoE design, combined with MLA, enables DeepSeek models to achieve top-tier performance while maintaining high training and inference efficiency.

Deliverable:

Submit a detailed technical paper synthesizing information from the provided sources. Your analysis should connect the architectural innovations (DSA, MLA, DeepSeekMoE) to the model's stated goals of high efficiency and superior performance in reasoning and agent tasks.

1.2 Project B: Train Your Own GPT Using the Muon Optimizer

This project involves a review of an alternative optimizer and a practical training exercise.

Required Components:

(1) Review the Muon Optimizer:

- **Note:** (See `https://kellerjordan.github.io/posts/muon/`).
- Your review should summarize the optimizer's proposed algorithm, its stated advantages over established optimizers (like AdamW), and its theoretical or empirical basis.
- Discuss its potential benefits and drawbacks for training modern neural networks, particularly transformer-based LLMs.

(2) Practical Training with nanoGPT:

- Use one of the referenced codebases (`https://github.com/KellerJordan/modded-nanogpt` or `https://github.com/Deveraux-Parker/nanoGPT_1GPU_SPEEDRUN`) or the original nanoGPT project to train a small-scale GPT model.
- Your goal is to set up a training run, potentially incorporating the Muon optimizer as a comparative experiment if feasible.
- Document your process, hyperparameters, and results.

Deliverable:

Submit a review paper covering the Muon optimizer and a separate report on your training exercise. The latter must include your training logs, the final model checkpoint, and a discussion of your results and observations.

Good Luck!